

Control laws in the Rattlesnake MIMO random environment

Algorithms, equations and flow charts · 6-12-8 system · 13 September 2026 · control_laws/

Five subjects: the four control laws compared on hardware, plus the achievable-response predictor that supplies the denominator every comparison is measured against. Each section gives the algorithm, the governing equations, a flow chart, the parameter string, and what was actually measured.

0. Common interface and notation

Rattlesnake loads a control law by explicit path with `importlib.util.spec_from_file_location` — there is no package context, so a law cannot use relative imports. A law is either a plain function called once per control cycle, a generator, or a class constructed at `INITIALIZE_PARAMETERS` (before system ID) whose `control()` is then called each cycle. The profile names the script, the function and the parameter string; the function type is inferred from the module.

Every law returns the same object: the commanded drive CPSD.

```
N    drives (6 here)      M    control channels (8)      F    lines (901 in band)

H(f)    M x N    frequency response function
X(f)    N x N    drive CPSD - what the law returns; must be PSD
Gxx(f)  M x M    specification CPSD (diagonal here)
Y(f) = H X H^H    predicted response CPSD
y_m     = diag(Y)_m the auto-spectra actually specified and scored
```

The constraint that shapes every law.

X must be a valid cross-spectral matrix, i.e. positive semidefinite. The set of achievable response auto-spectra is therefore $D = \{ \text{diag}(H X H^H) : X \succeq 0 \}$, the image of the PSD cone under a linear map. When $M > N$ — eight control channels from six shakers — D is a proper subset of the positive orthant, so a perfectly reasonable flat specification can be partly unreachable. No law gets outside D . How each law respects positive semidefiniteness is the main thing that distinguishes them.

1. match_trace_pseudoinverse

function · control_laws.py

The baseline, and the only law in the set with a long validation history. One open-loop pseudoinverse solve at startup, then a single real scalar per frequency line for the rest of the test.

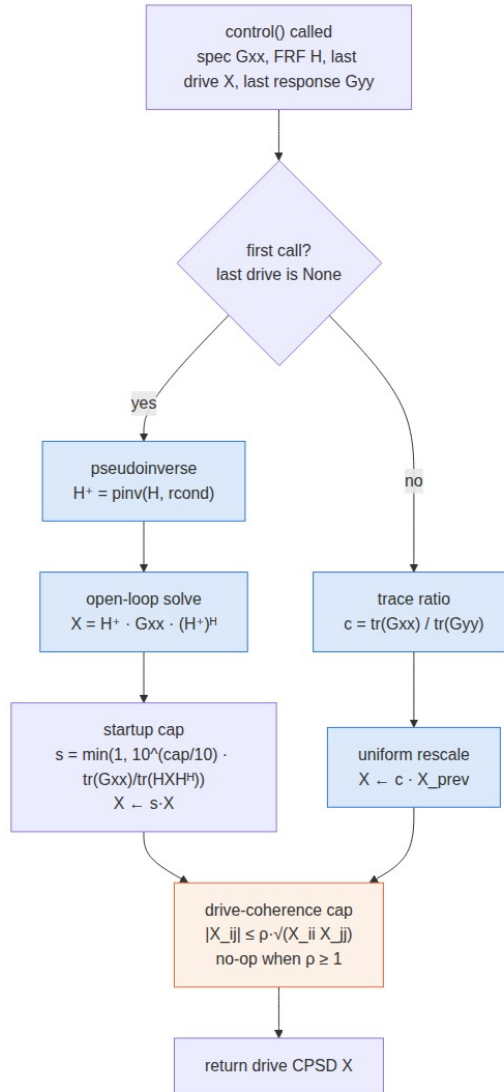
Algorithm

```
first call - open loop
H+ = pinv(H, rcond)
X = H+ Gxx (H+)^H
s = min(1, 10^(cap/10) * tr(Gxx) / tr(H X H^H))  <- startup guard
X <- s * X

every call after that - closed loop on total power only
c(f) = tr(Gxx(f)) / tr(Gyy(f))
X(f) <- c(f) * X_prev(f)
```

The update is, in log-power terms, a **pure discrete integrator with gain 1** on the log-power error: writing $e(k) = \log \text{tr}(G_{xx}) - \log \text{tr}(G_{yy}(k))$, the update is exactly $\log X(k) = \log X(k-1) + e(k-1)$. There is no proportional term, and the error it integrates arrives through whatever CPSD averaging is configured — a pure integrator fed a lagged error, the classic slow-to-settle, overshoot-prone configuration. Simulation at averaging coefficient 0.10 gave ~53 cycles and ~103 % peak overshoot to settle within 5 %.

Why it is safe: a uniform positive real scalar can never turn a valid drive CPSD into an invalid one, and after startup the FRF is never inverted again. **Why it is limited:** if channel 3 is 5 dB high and channel 5 is 5 dB low, the trace is correct and the law does nothing at all.



match_trace_pseudoinverse. The only per-cycle computation is one scalar per frequency line.

Parameters

Pos	Name	Default	Meaning
1	rcond	1e-15	Pseudoinverse conditioning cutoff. Startup solve only. 1e-3 used in this study.
2	max_drive_coherence	1.0	Cap on pairwise drive coherence. 1.0 is a true no-op — the CPSD is returned unchanged.
3	startup_test_level_cap_db	-9.0	Ceiling on the first command's predicted response trace, relative to the spec trace.

2. optimal_diagonal_control

class · optimal_diagonal_control.py

Chooses the drive CPSD's cross-terms to minimise diagonal error directly, rather than fixing them from an independent-drive survey. Per frequency line it solves a genuine semidefinite program.

The per-bin SDP

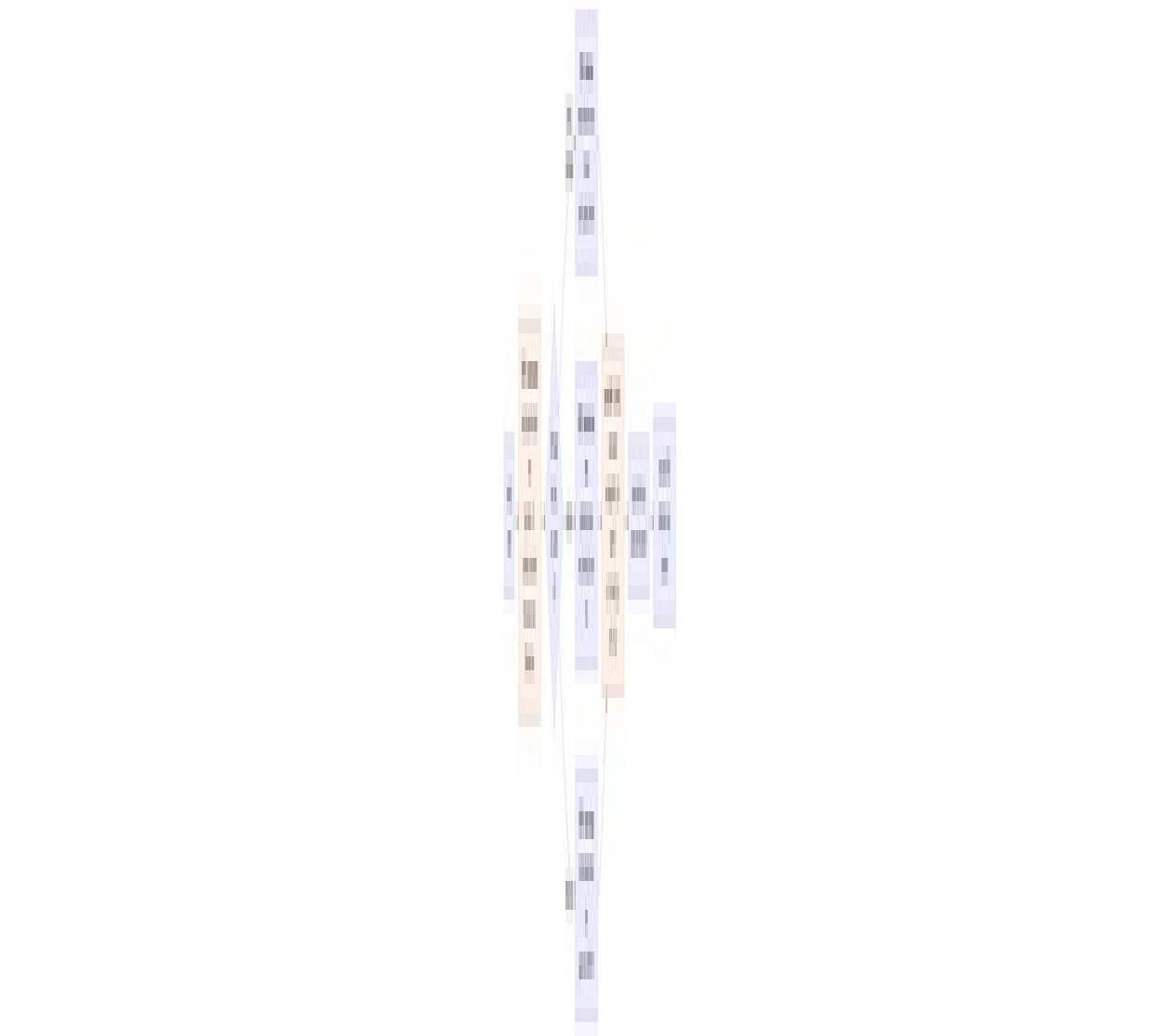
```
minimize_X    || diag(H X H^H) - y_target ||^2 + reg * ||X||_F^2
subject to    X >= 0    (positive semidefinite)
              |X_ij| <= rho * sqrt(X_ii * X_jj)    for all i != j
```

Both constraints are convex — the coherence one is $|\text{affine}| \leq \text{concave}$ — so this is a true convex program with no manifold optimisation, solved with CLARABEL and warm-started between calls.

The budgeted scheduler

Solving every line up front costs seconds. Instead the law runs a five-step schedule per call, spending a fixed budget of `max_bins_per_update` SDP solves where they do most good, so control can begin on iteration 1 with a full-spectrum result:

1. **Instant baseline.** Every bin gets Rattlesnake's closed-form buzz solve — H+ synthesis with cross-terms from a survey if available, zero if not. Effectively free.
2. **Targeted refinement.** The worst-predicted-error bins above `error_threshold_db` get the full SDP, replacing their buzz solution.
3. **Progressive.** Repeats every call, chipping away until the band is refined or good enough.
4. **FRF drift.** Already-refined bins whose H moved past `frf_update_threshold` are re-solved with priority, capped at half the budget so they cannot starve step 2.
5. **Stale re-triage.** Leftover budget re-checks already-refined bins against the current H rather than H-drift magnitude — without this, once every bin is refined step 2 never runs again and a degrading bin is orphaned on its stale solution forever.



optimal_diagonal_control. The budget is spent worst-error-first, with drifted bins taking priority.

The coherence cap reaches only SDP-refined bins.

The buzz baseline is left uncapped on purpose — it draws its cross-terms from an independent-drive survey, so it has no dependent-drive problem to begin with. With `max_bins_per_update` = 20 across 901 lines, most bins carry the uncapped baseline. This is the most likely explanation for the measured null result: turning the cap off changed the overall error by 0.008 dB.

Parameters

Pos	Name	Default	Meaning
1	<code>reg</code>	<code>1e-6</code>	Tikhonov weight on the Frobenius norm of X .
2	<code>frf_update_threshold</code>	<code>0.05</code>	Relative Frobenius change in H that triggers re-solving a refined bin.
3	<code>max_bins_per_update</code>	<code>20</code>	Hard cap on SDP solves per call.
4	<code>error_threshold_db</code>	<code>1.0</code>	Do not refine a bin already this close under buzz.

Pos	Name	Default	Meaning
5	max_drive_coherence	0.95	Coherence cap, applied inside the SDP. 1.0 disables it.

3. optimal_diagonal_control_fast

subclass · optimal_diagonal_control_fast.py

Identical baseline and scheduler, inherited unchanged. The only override is how a single bin is solved: the SDP is replaced by an unconstrained Burer-Monteiro factored solve whenever that is safe.

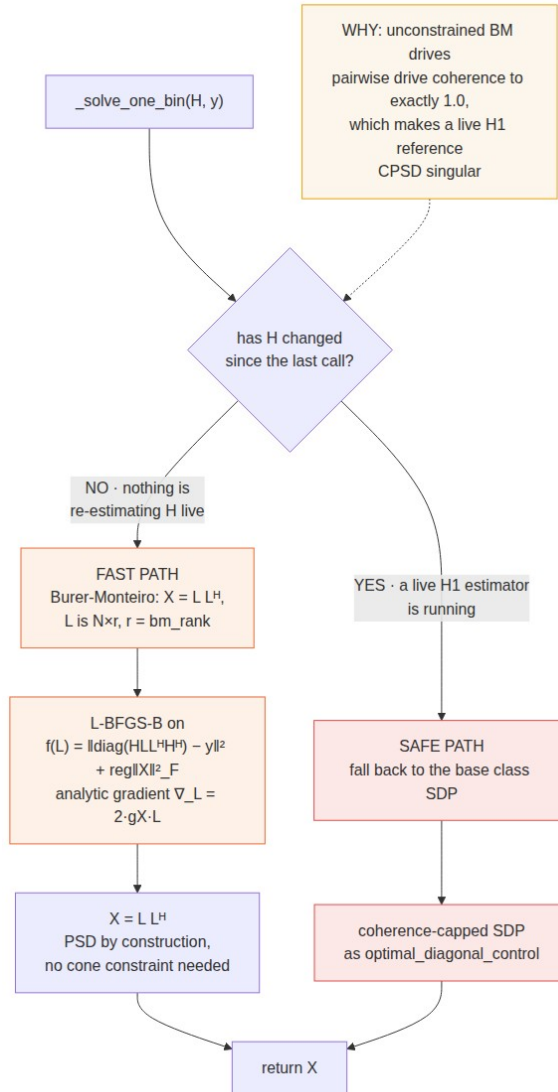
```
X = L L^H,    L in C^(N x r),    r = bm_rank

minimize_L || diag(H L L^H H^H) - y ||^2 + reg ||X||_F^2
grad_X = 2 H^H (resid * H) + 2 reg X      grad_L = 2 grad_X L
L-BFGS-B, analytic gradient, maxiter 50
```

Factoring X as $L L^H$ makes positive semidefiniteness structural — the parametrisation is unconstrained, so no cone projection is needed and the solve is far cheaper. Benchmarked at a 12-drive/30-response scale: SDP ~105–119 ms/bin against ~5.4–5.7 ms/bin for the factored solve, roughly 18–20x faster, matching SDP accuracy to a mean of -0.02 dB.

The safety rule, and why it disabled the fast path here.

Left unconstrained, the factored solve drives pairwise drive coherence to exactly 1.0 in essentially every bin — perfectly rank-deficient drives, which would make a live H1 estimator's reference CPSD matrix singular. Rather than trying to detect the "Update Transfer Function During Control" setting, which the call signature does not expose, the law watches the safety-relevant property directly: has H changed since the last call? If it has, every solve for the rest of that call uses the safe capped SDP instead. In this study FRF update was on, so H changed every call and the fast path never engaged — runs 05 and 06 are repeats of 03 and 04.



optimal_diagonal_control_fast. The branch at the top is the whole law; everything else is inherited.

Parameters

The same five as `optimal_diagonal_control`, plus:

Pos	Name	Default	Meaning
6	<code>bm_rank</code>	4	Factorisation rank r . With $M = 8$ the threshold below which spurious local minima generically appear is $\sqrt{\text{rank}(M)} \sim 4$, so the default sits exactly on it; 6 was used in this study.

4. match_diagonal_congruence

class · diagonal_congruence_control.py

Generalises match_trace from one scalar to N independent knobs, while keeping the structural property that makes match_trace safe.

The generalisation

```
match_trace:  X <- c * X                                (one real scalar)
this law:     X <- D X D,  D = diag(e^u1 ... e^uN),  u real
```

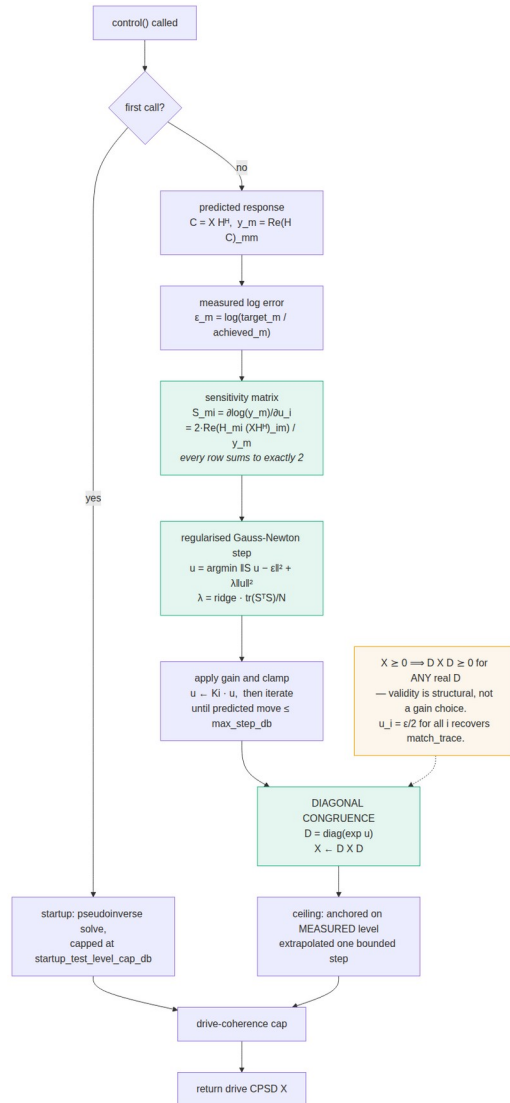
A diagonal congruence preserves positive semidefiniteness structurally: if $X \succeq 0$ then $D X D \succeq 0$ for any real diagonal D . So exactly as with match_trace, no choice of gains — however wrong the FRF, however noisy the measurement — can command a physically invalid drive. match_trace is the special case $D = \sqrt{c} I$.

Choosing the gains

```
y_m = (H X H^H)_mm                                predicted response
S_mi = d log(y_m) / d u_i = 2 Re( H_mi (X H^H)_im ) / y_m
eps_m = log( target_m / achieved_m )                measured log error

u = argmin || S u - eps ||^2 + lambda ||u||^2
lambda = ridge * tr(S^T S) / N
one Gauss-Newton step on the log-domain problem,
restricted to diagonal congruence
```

Every row of S sums to exactly 2, because scaling all gains together by t scales X by $e^{(2t)}$ and hence y by $e^{(2t)}$. The consequence is that when the error is the same on every channel the solution is $u_i = \text{eps}/2$ for all i — *the law reproduces match_trace's update exactly*. It degrades gracefully to the law it generalises.



match_diagonal_congruence. The congruence step is what guarantees validity; the Gauss-Newton step only chooses how far to move.

max_step_db bounds the response, not the gains.

An early version clamped u directly, which does not bound the outcome: rows of S sum to 2 but individual entries can be large and opposite-signed, so a 3 dB clamp on u produced a 38 dB swing in response. The clamp is now enforced on the exact predicted per-channel response move, iteratively, up to six passes.

Parameters

Pos	Name	Default	Meaning
1	rcond	1e-3	Pseudoinverse cutoff for the startup solve.
2	max_drive_coherence	0.95	Drive coherence cap; 1.0 disables.
3	startup_test_level_cap_db	-9.0	Startup ceiling, as match_trace.
4	Ki	0.5	Gain applied to the Gauss-Newton step.

Pos	Name	Default	Meaning
5	max_step_db	3.0	Bound on the predicted per-channel response move per cycle.
6	ridge	0.1	Regularisation lambda, relative to $\text{tr}(S^T S)/N$.
7	resonance_sensitivity	0.5	Half-power resonance detection sensitivity.
8	ceiling_db	12.0	Ceiling anchored on measured level, extrapolated one bounded step.
9	projected_target_file	none	Optional .npz whose target_diag replaces the raw specification as the target. Not used in this study — all four laws aimed at the same raw specification.

5. achievable_response

not a control law · achievable_response.py

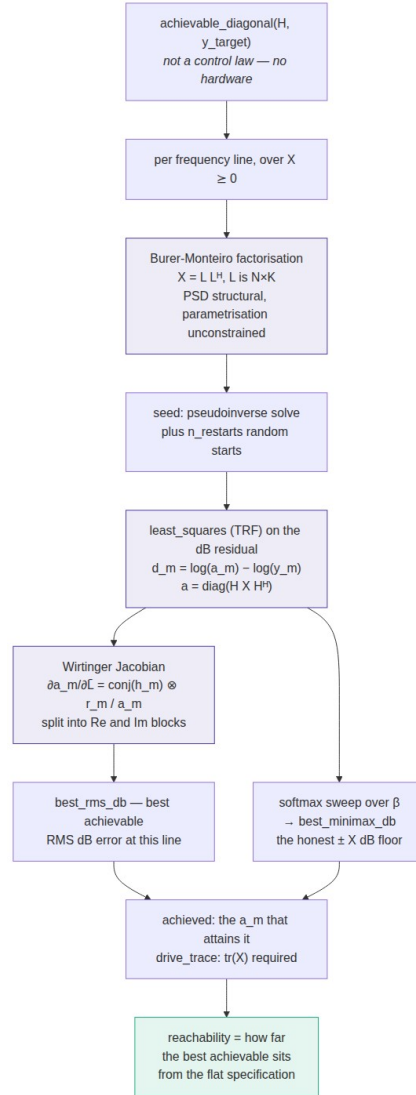
The pseudo control law. It drives no hardware. Given an FRF it answers the question every comparison needs a denominator for: how much of my response error is the control law's fault, and how much is simply not reachable with this structure and these shakers?

per frequency line, over $X \geq 0$:

```
best_rms_db      min sqrt( mean_m (10 log10(a_m / y_m))^2 )
best_minimax_db  min max_m |10 log10(a_m / y_m)|
                  the honest "+/- X dB" floor
achieved         the a_m = diag(H X H^H) that attains it
drive_trace      tr(X) required - feasibility against shaker limits
```

Solved by the same Burer-Monteiro factorisation the fast law uses, $X = L L^H$, but minimising the residual in the dB domain rather than the linear one — dB error is the quantity a vibration test is judged by, and a linear residual over-weights loud channels. The Jacobian is built with Wirtinger calculus:

```
d_m = log(a_m) - log(y_m)                residual
da_m/dL_bar = conj(h_m) outer r_m / a_m
J = [ Re(G) | Im(G) ]                    real and imaginary blocks, 2NK columns
least_squares(method='trf') - not 'lm', which refuses  $m < n$ 
```



achievable_response. The output is a bound, not a command — but it is what makes "how good is this law" a well-posed question.

Parameters

Name	Default	Meaning
rank	N	Factorisation rank K. N = 6 here, above the $\sqrt{2M} \sim 4$ threshold.
n_restarts	2	Random restarts beyond the pseudoinverse seed. restart_spread_db reports landscape ruggedness, not unreliability.
rcond	1e-8	Conditioning cutoff for the starting point only; does not constrain the answer.
restrict_rcond	none	Truncates the drive subspace — the honest way to ask "what is achievable with well-conditioned drives?" 1e-3 costs 0.74 dB while saving four and a half orders of magnitude of drive power.
compute_minimax	True	Also solve the minimax objective via a softmax sweep over beta.
max_nfev	400	Function-evaluation budget per line.

6. Lineage and measured results

Each law solves the same problem with a different guarantee of validity, and that choice is what determines both its power and its limits:

Law	How $X \succeq 0$ is guaranteed	DOF per line	rms dB	% out	drive
match_trace_pseudoinverse	uniform positive scalar	1	8.22	27.5	0.12
optimal_diagonal_control	explicit SDP cone constraint	N^2 (full X)	5.62	11.2	0.075
optimal_diagonal_control_fast	structural, $X = L L^H$	$2Nr$	5.64	10.9	0.076
match_diagonal_congruence	structural, $D X D$	N	5.31	19.8	0.047

Results are the cap-on runs from the 12 September comparison, linear plant, against the flat specification. Scored instead against what is achievable, congruence leads at 3.49 dB against optimal_diagonal's 4.00 dB.

Open defects.

Neither optimal-diagonal law has a startup level cap — match_trace and congruence both take startup_test_level_cap_db, but the optimal-diagonal laws solve against the full specification and command it immediately. The fast law's central claim, that Burer-Monteiro at rank 6 matches the SDP, remains untested on this system because FRF update disabled its fast path.

Algorithms read from the implementations in control_laws/ at commit e6f961d6. Flow charts are Mermaid; sources are committed alongside this document so they stay editable as text.